

Examen Parcial: Estructuras de Datos en Viajes Temporales

Instrucciones Generales

- Este examen está diseñado para evaluar su comprensión y aplicación de los conceptos de estructuras de datos, incluyendo TADs, listas enlazadas, colas, árboles, árboles binarios y árboles binarios de búsqueda.
- Responda cada ejercicio con claridad y justifique cuando sea indicado.
- El examen se basa en viajes en el tiempo, utilice su creatividad para implementar lo pedido.

Ejercicio 1: Implementación de TADs

Enunciado

En esta simulación de viajes temporales, cada viajero tiene un dispositivo de salto temporal. Para este ejercicio deberá implementar dos clases en Python: **ViajeroTemporal** y **DispositivoTemporal**.

Clases a Implementar

1. Clase **ViajeroTemporal**:

◦ **Atributos:**

- **nombre**: (*str*) Nombre del viajero temporal.
- **epoca_origen**: (*str*) Época de origen del viajero (por ejemplo, "Futuro", "Presente", "Pasado").
- **saltos_realizados**: (*int*) Número de saltos temporales realizados (debe estar entre 0 y 1000).

◦ **Métodos:**

- **__init__()**: Constructor que inicializa los atributos del viajero temporal.
- **realizar_salto()**: Método que incrementa el número de saltos realizados en 1, siempre y cuando no supere los 1000 saltos.
- **__str__()**: Método que devuelve una representación en cadena del viajero temporal en el formato: "Nombre: [nombre], Época de Origen: [epoca_origen], Saltos Realizados: [saltos_realizados]"

2. Clase **DispositivoTemporal**:

◦ **Atributos:**

- **modelo**: (*str*) Modelo del dispositivo temporal.
- **viajeros**: (*lista*) Lista que contiene instancias de la clase **ViajeroTemporal**.

◦ **Métodos:**

- **__init__()**: Constructor que inicializa el modelo del dispositivo y crea una lista vacía para los viajeros.
- **agregar_viajero()**: Método que agrega un viajero al dispositivo. Si ya hay 5 viajeros en el dispositivo, debe mostrar un mensaje indicando que no se pueden tener más

viajeros.

- `mostrar_viajeros()`: Método que imprime todos los viajeros del dispositivo utilizando el método `__str__` de la clase `ViajeroTemporal`.
- `promedio_saltos()`: Método que calcula y devuelve el promedio de saltos realizados por los viajeros en el dispositivo. Si no hay viajeros, debe devolver 0.

Ejercicio 2: Recursividad en Viajes Temporales

Enunciado

Los viajeros temporales a menudo necesitan buscar eventos específicos en la línea temporal. Para este ejercicio implementar una función recursiva que simula la búsqueda de un evento histórico en un árbol de eventos temporales.

Contexto

Suponga que tiene un árbol binario donde cada nodo representa un evento histórico, cada nodo tiene un nombre y puede tener hasta dos hijos (uno a la izquierda y otro a la derecha). La búsqueda de un evento específico se puede realizar de manera recursiva verificando si el evento se encuentra en el nodo actual o en uno de sus subárboles.

Clases a Implementar

1. Clase `EventoTemporalNode`:

◦ Atributos:

- `nombre`: (*str*) Nombre del evento histórico.
- `izquierda`: (*EventoTemporalNode*) Hijo izquierdo (puede ser `None`).
- `derecha`: (*EventoTemporalNode*) Hijo derecho (puede ser `None`).

◦ Métodos:

- `__init__(self, nombre: str)`: Constructor que inicializa el nombre del evento y establece los hijos izquierdo y derecho como `None`.

2. Función Recursiva `buscar_evento`:

◦ Parámetros:

- `nodo`: (*EventoTemporalNode*) Nodo actual del árbol donde se realiza la búsqueda.
- `nombre_evento`: (*str*) Nombre del evento que se busca.

◦ Retorno:

- Devuelve `True` si el evento es encontrado en el árbol, y `False` si no se encuentra.

◦ Lógica:

- Si el nodo actual es `None`, devuelve `False`.
- Si el nombre del nodo actual coincide con `nombre_evento`, devuelve `True`.
- Llama recursivamente a sí misma para buscar en el hijo izquierdo y luego en el hijo derecho.

Ejercicio 3: Sistema de Paradojas Temporales

Enunciado

Cada viaje en el tiempo puede generar paradojas que deben ser resueltas en orden. Para gestionarlas, implementar un sistema utilizando una **cola** (queue) implementada con una lista enlazada, que permita llevar un registro de las paradojas en el orden en que fueron creadas, ya que la más antigua es la que se resolverá primero.

Contexto

La cola se utilizará para almacenar las paradojas generadas por los viajes en el tiempo. Cuando se detecta una nueva paradoja, se agrega a la cola. Los Guardianes del Tiempo resolverán las paradojas en el orden en que fueron creadas.

Clases a Implementar

1. Clase **Paradoja**:

- **Atributos:**

- **descripcion**: (*str*) Descripción de la paradoja temporal.

- **Métodos:**

- **__init__(self, descripcion: str)**: Constructor que inicializa la descripción de la paradoja.

2. Clase **Nodo**:

- **Atributos:**

- **dato**: (*Paradoja*) La paradoja almacenada en el nodo.
- **siguiente**: (*Nodo*) Referencia al siguiente nodo en la lista enlazada.

- **Métodos:**

- **__init__(self, paradoja: Paradoja)**: Constructor que inicializa el nodo con una paradoja y establece el siguiente como None.

3. Clase **ColaParadojas**:

- **Atributos:**

- **frente**: (*Nodo*) Referencia al primer nodo de la cola.
- **final**: (*Nodo*) Referencia al último nodo de la cola.

- **Métodos:**

- **__init__(self)**: Constructor que inicializa la cola vacía.
- **esta_vacia(self)**: Método que devuelve True si la cola está vacía, False en caso contrario.
- **encolar(self, paradoja: Paradoja)**: Método que agrega una paradoja al final de la cola.
- **desencolar(self)**: Método que elimina y devuelve la paradoja del frente de la cola. Si la cola está vacía, debe devolver un mensaje indicando que no hay paradojas para resolver.
- **mostrar_paradojas(self)**: Método que imprime todas las paradojas en el orden en que fueron encoladas.

Ejercicio 4: Diseño Teórico de un Juego de Viajes en el Tiempo

Enunciado

Imagine que debe desarrollar un juego de viajes en el tiempo.

Este juego incluirá elementos como saltos temporales, alteración de eventos históricos, encuentros con versiones pasadas o futuras de personajes, resolución de paradojas y protección de la línea temporal. Para cada uno de estos componentes deberá explicar, qué estructuras de datos utilizaría y por qué; considerando la eficiencia y eficacia en cada caso.

Componentes mínimos del Juego a considerar

1. **Salto Temporal**
2. **Alteración de Eventos Históricos**
3. **Encuentros con Versiones Alternativas de Personajes**
4. **Resolución de Paradojas**
5. **Protección de la Línea Temporal**

Consideraciones Finales

Al diseñar este juego de viajes en el tiempo es fundamental, considerar no solo, la estructura de datos más adecuada para cada componente sino también cómo estas estructuras interactúan entre sí. La elección de estructuras eficientes además de mejorar el rendimiento del juego también proporciona una mejor experiencia al jugador al permitir interacciones fluidas y rápidas.

Criterios de Evaluación

1. Corrección y Complejidad del Código (30 puntos)

- Implementación correcta de las clases y métodos solicitados.
- Uso adecuado de las estructuras de datos.
- Complejidad algorítmica y eficiencia del código.

2. Claridad y Legibilidad (20 puntos)

- Estructura y organización del código.
- Uso de comentarios que expliquen la lógica y el propósito del código.
- Nombres de variables y métodos descriptivos.

3. Justificación Teórica (40 puntos)

- Explicación clara y lógica de las elecciones de estructuras de datos en el ejercicio teórico.
- Consideración de la eficiencia y eficacia en el uso de las estructuras seleccionadas.
- Integración creativa y coherente de los elementos de viajes en el tiempo en el diseño del juego.

4. Manejo de Casos Especiales (10 puntos)

- Capacidad para manejar errores y condiciones especiales en la implementación.
- Robustez del código frente a entradas inesperadas o situaciones límite.

Puntaje Total Viajes en el Tiempo: 100 paradojas

Dato

Los estudiantes disponen de **4 horas** para resolver el examen. Se recomienda gestionar bien el tiempo para abordar cada ejercicio con la profundidad necesaria.

¡Buena suerte, viajeros del tiempo!